Module 3: 2D Defect Evolution in Silicon

Diffusion-Reaction Model, Numerics, and Verification

Project context. This note defines the first two-dimensional implementation target for Module 3 in `RadiationEffects_proj2`. The purpose of Module 3 is to evolve radiation-induced defect populations in silicon after their creation by Module 1, so that later modules can compute defect-modified electrostatics, temperature response, and carrier transport.

Project:	Radiation-Induced Defect Evolution and Device Degradation in Silicon
Document:	`module3_2d_defect_evolution.tex/.pdf`
Role in roadmap:	First 2D physics solver on the MATLAB side
Implementation level:	Structured 2D Cartesian baseline before more general discretizations

March 30, 2026

1 Purpose and scope

Module 3 is the first post-radiation evolution model in the project. Module 1 tells us where radiation deposited energy and, after additional modeling assumptions, where defects were created. Module 3 answers the next question: once defects exist inside silicon, how do they move and transform with time?

The first implementation target is intentionally narrow and rigorous:

- two spatial dimensions, x and y, on a structured Cartesian grid,
- one defect concentration field $C(x, y, t)$,
- diffusion of defects through the silicon,
- first-order annealing as the simplest reaction channel,
- zero-normal-flux boundary conditions for a closed-material baseline, and
- verification cases that isolate diffusion and annealing separately.

This note is written so that the physics and the MATLAB implementation stay tightly connected. The equations are stated first, then reduced to the specific numerical form we will actually solve.

2 Physical picture

A radiation event can displace atoms, create vacancies and interstitials, and generate more complicated defect structures. Immediately after the event, the damage is not yet a final material state. Defects can:

- migrate through the lattice,
- recombine with complementary defects,
- form clusters,
- dissociate from larger complexes, or
- anneal away as the system moves toward a lower-energy configuration.

The full physics can become chemically and crystallographically rich. The first 2D solver deliberately starts below that complexity. It asks for the evolution of a coarse-grained defect concentration field, not the trajectory of each atom. In that coarse-grained description, the two most important mechanisms to represent first are:

1. **Diffusion:** defects spread from regions of high concentration toward regions of lower concentration.
2. **Annealing:** defects disappear from the tracked population at a rate controlled by local kinetics.

That gives a model that is simple enough to verify analytically, but still physically meaningful as a bridge between the radiation source map and later device-level degradation calculations.

3 Governing equation

3.1 Conservation statement

Let $C(x, y, t)$ denote the concentration of one defect species. For a small material area element, conservation requires

$$\frac{\partial C}{\partial t} = -\nabla \cdot \boldsymbol{J} + R, \tag{1}$$

where $\boldsymbol{J}$ is the defect flux vector and R is the net local reaction term.

Equation (1) is the cleanest starting point because it states the physics directly:

- the concentration changes because defects flow in or out of a region, and/or
- the concentration changes because defects are created or removed by local reactions.

3.2 Diffusive flux

For the first implementation, we assume Fickian diffusion:

$$\boldsymbol{J} = -D\,\nabla C, \tag{2}$$

where D is the defect diffusion coefficient.

Substituting Equation (2) into Equation (1) gives

$$\frac{\partial C}{\partial t} = \nabla \cdot (D\,\nabla C) + R. \tag{3}$$

If D is spatially uniform, this reduces to

$$\frac{\partial C}{\partial t} = D\nabla^2 C + R. \tag{4}$$

In 2D Cartesian coordinates,

$$\frac{\partial C}{\partial t} = \frac{\partial}{\partial x}\left(D\frac{\partial C}{\partial x}\right) + \frac{\partial}{\partial y}\left(D\frac{\partial C}{\partial y}\right) + R. \tag{5}$$

For constant D,

$$\frac{\partial C}{\partial t} = D\left(\frac{\partial^2 C}{\partial x^2} + \frac{\partial^2 C}{\partial y^2}\right) + R. \tag{6}$$

3.3 First-order annealing model

The first reaction model is first-order annealing:

$$R = -k_{\text{ann}} C, \tag{7}$$

where k_{ann} is the annealing rate constant.

Then the governing equation becomes

$$\boxed{\frac{\partial C}{\partial t} = D\left(\frac{\partial^2 C}{\partial x^2} + \frac{\partial^2 C}{\partial y^2}\right) - k_{\text{ann}} C.} \tag{8}$$

This is the baseline PDE for the first 2D Module 3 implementation.

4 Why this model is the right first step

This first PDE is not the final defect-chemistry model. It is the correct first step because it gives all of the following at once:

- a direct way to consume a 2D damage map from Module 1,
- a physically meaningful time evolution of damage,
- a field $C(x, y, t)$ that later modules can map into charge, recombination, and thermal effects,
- analytic limiting cases for verification, and
- a clean extension path to multiple defect species C_i, nonlinear reactions R_i, and temperature-dependent kinetics.

In other words, Equation (8) is simple enough to debug and strong enough to become infrastructure.

5 Boundary and initial conditions

5.1 Boundary conditions

The first implementation uses zero-normal-flux boundary conditions:

$$\boldsymbol{n} \cdot \boldsymbol{J} = 0, \tag{9}$$

where $\boldsymbol{n}$ is the outward unit normal vector at the boundary.

Using $\boldsymbol{J} = -D\nabla C$, this becomes

$$\boldsymbol{n} \cdot \nabla C = 0. \tag{10}$$

Physically, this means defects do not leave the computational domain through the outer boundary in the baseline case.

5.2 Initial condition

The initial condition can come from either a synthetic verification profile or a processed Geant4-derived map:

$$C(x, y, 0) = C_0(x, y). \tag{11}$$

For verification, two convenient choices are:

- a 2D Gaussian cloud, and
- a spatially uniform field.

6 Analytic limiting cases for verification

6.1 Pure annealing

If diffusion is switched off, so that $D = 0$, Equation (8) reduces to

$$\frac{\partial C}{\partial t} = -k_{\text{ann}} C. \tag{12}$$

At every spatial point, the solution is

$$C(x, y, t) = C(x, y, 0)\, e^{-k_{\mathrm{ann}} t}. \tag{13}$$

If the initial field is uniform, the field stays uniform for all time and decays exponentially. This is an excellent code test because any spatial structure that appears is numerical error.

6.2 Pure diffusion

If annealing is switched off, so that $k_{\mathrm{ann}} = 0$, Equation (8) becomes

$$\frac{\partial C}{\partial t} = D \nabla^2 C. \tag{14}$$

For a localized initial cloud in an effectively large domain, the defect distribution broadens with time while conserving total defect amount,

$$M(t) = \iint C(x, y, t)\, \mathrm{d}x\, \mathrm{d}y = \text{constant}, \tag{15}$$

provided the boundary flux is zero.

A Gaussian profile remains Gaussian under pure diffusion, with a width that grows in time. Even if the finite computational domain and boundaries modify the exact analytic shape, the qualitative signatures are robust:

- peak concentration decreases,
- cloud width increases,
- total mass remains nearly constant.

7 Temperature dependence and future coupling

Even though the first version may hold temperature fixed, Module 3 is fundamentally temperature sensitive. In many defect processes, both the diffusion coefficient and the annealing rate are thermally activated. A common form is Arrhenius-like behavior:

$$D(T) = D_0 \exp\left(-\frac{E_D}{k_B T}\right), \tag{16}$$

$$k_{\mathrm{ann}}(T) = k_0 \exp\left(-\frac{E_a}{k_B T}\right), \tag{17}$$

where D_0 and k_0 are prefactors, E_D is a diffusion activation energy, E_a is an annealing activation energy, and k_B is Boltzmann's constant.

This matters for the project roadmap because Module 4 will provide the temperature field $T(x, y, t)$. Once that happens, Module 3 can be upgraded from constant-parameter kinetics to locally temperature-dependent kinetics without changing the overall conservation form of the solver.

8 Numerical implementation strategy

8.1 Why a structured 2D Cartesian baseline

The project-level roadmap refers to diffusion-reaction finite element modeling, which is a reasonable long-term direction for arbitrary geometries and heterogeneous materials. For the first implementation, however, a structured 2D Cartesian discretization is the better engineering choice because it is:

- easier to implement correctly,
- easier to verify against analytic limits,
- easier to visualize and debug, and
- fully adequate for the first rectangular silicon-domain studies.

So the immediate implementation should be understood as a **2D structured baseline** that establishes the physics and software interface before a more general discretization is introduced.

8.2 Grid

Let the computational domain be discretized by N_x points in the x-direction and N_y points in the y-direction, with spacings

$$\Delta x = x_{i+1} - x_i, \qquad \Delta y = y_{j+1} - y_j. \tag{18}$$

The discrete approximation to the concentration field is written as $C^n_{i,j}$, meaning the value at grid location (i, j) and time step n.

8.3 Spatial discretization

For uniform D, the standard second-order central-difference Laplacian is

$$\nabla^2 C\big|_{i,j} \approx \frac{C_{i+1,j} - 2C_{i,j} + C_{i-1,j}}{\Delta x^2} + \frac{C_{i,j+1} - 2C_{i,j} + C_{i,j-1}}{\Delta y^2}. \tag{19}$$

With mirrored ghost-cell logic for the Neumann boundary condition, the derivative normal to the boundary is forced to zero.

8.4 Time stepping

The first implementation can use an explicit forward-Euler update,

$$C^{n+1}_{i,j} = C^n_{i,j} + \Delta t \left[D\, \nabla^2_h C^n_{i,j} - k_{\text{ann}} C^n_{i,j} \right], \tag{20}$$

where ∇^2_h denotes the discrete Laplacian.

This is straightforward and good for verification, but it carries a diffusion stability restriction. A practical estimate is

$$\Delta t \lesssim \frac{1}{2D\left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}\right)}. \tag{21}$$

Equation (21) is crucial. When the grid is refined, the stable explicit time step decreases rapidly. That is acceptable for the first verification stage, but it is also the reason the project should later consider:

- implicit diffusion solves,
- operator splitting, or
- sparse linear-system formulations.

8.5 Mass and positivity checks

For the pure-diffusion case with zero-flux boundaries, the discrete total mass should remain approximately constant:

$$M^n = \sum_{i,j} C_{i,j}^n \, \Delta x \, \Delta y. \tag{22}$$

For the pure-annealing case, the spatially uniform initial state should remain uniform and follow Equation (13). In practice, it is also reasonable to clamp tiny negative values caused by explicit time stepping and roundoff, but that should be treated as a numerical safeguard, not as a substitute for a stable update scheme.

9 Recommended file map for the first 2D Module 3 implementation

The following file map matches the coding direction already discussed for the MATLAB side:

File or folder	Purpose
`matlab/main_module3_2d_defect_evolution.m`	Main driver for 2D defect evolution runs
`matlab/cases/case_module3_gaussian_diffusion_2d.m`	Verification case for pure diffusion from a localized cloud
`matlab/cases/case_module3_pure_annealing_2d.m`	Verification case for spatially uniform exponential decay
`matlab/src/grid/make_grid_2d.m`	Grid generation for the 2D Cartesian domain
`matlab/src/module3_defects/initialize_defect_field_2d.m`	Builds the initial concentration field
`matlab/src/module3_defects/laplacian_2d_neumann.m`	Discrete Laplacian with zero-normal-gradient boundaries
`matlab/src/module3_defects/step_defect_diffusion_reaction_2d.m`	One time step of the diffusion-reaction update
`matlab/src/io/save_history_2d.m`	Stores snapshots, total mass, and extrema
`matlab/src/plot/plot_field_2d.m`	Visualizes the 2D concentration field
`matlab/tests/test_module3_gaussian_diffusion_2d.m`	Test wrapper for the diffusion verification case
`matlab/tests/test_module3_pure_annealing_2d.m`	Test wrapper for the annealing verification case

10 Verification plan

The first 2D Module 3 implementation should not begin with full Geant4-coupled runs. It should begin with controlled tests.

10.1 Verification Case A: 2D Gaussian diffusion

Setup:

- initial condition: localized 2D Gaussian,
- diffusion coefficient $D > 0$,
- annealing rate $k_{\text{ann}} = 0$,

- zero-flux boundaries.

Expected behavior:

- the peak concentration decreases,
- the profile broadens,
- the total defect mass remains nearly constant.

10.2 Verification Case B: uniform-state preservation

Setup:

- initial condition: $C(x, y, 0) = C_0$,
- diffusion coefficient $D > 0$,
- annealing rate $k_{\text{ann}} = 0$.

Expected behavior:

- the field remains uniform to numerical precision.

This test is especially useful for catching boundary-condition mistakes or indexing bugs in the discrete Laplacian.

10.3 Verification Case C: pure annealing

Setup:

- initial condition: uniform or nonuniform,
- diffusion coefficient $D = 0$,
- annealing rate $k_{\text{ann}} > 0$.

Expected behavior:

- every grid point decays as $e^{-k_{\text{ann}}t}$,
- a uniform field stays uniform,
- numerical results can be directly compared against Equation (13).

10.4 Verification Case D: imported damage map

Only after Cases A-C are behaving correctly should the code ingest a processed 2D damage map derived from Module 1. At that stage, the key checks are not analytic exactness, but rather:

- consistent data import,
- sensible physical smoothing and decay,
- reproducible saved history, and
- readiness for coupling to Module 2 and Module 4.

11 How Module 3 connects to the rest of the project

Module 3 is not an isolated math exercise. Its output feeds directly into later modules:

- **Module 2:** defect concentration can be mapped into defect charge or trap density, which changes the space-charge field and electrostatic potential.
- **Module 4:** temperature modifies diffusion and annealing rates, so the thermal solver supplies $T(x, y, t)$ back into Module 3.
- **Module 5:** evolving defect populations alter recombination, trapping, and carrier lifetime, which changes electrical response.
- **Module 6:** the full coupled solver iterates among these effects self-consistently.

That is why building Module 3 first is strategically correct. It establishes the first dynamic material-state variable in the MATLAB side of the project.

12 Planned extensions beyond the first solver

Once the baseline 2D single-species solver is working and verified, the next upgrades are natural:

1. multiple defect species C_i,
2. nonlinear reaction channels $R_i(C_1, C_2, \ldots)$,
3. temperature-dependent $D(T)$ and $k_{\text{ann}}(T)$,
4. spatially varying material parameters,
5. implicit or semi-implicit time integration, and
6. more general geometries and discretizations.

In short, the first 2D solver is not a detour. It is the foundation on which the richer defect-physics model will be built.

13 Variables and symbols

Symbol	Name	Meaning
$C(x, y, t)$	defect concentration	Concentration of the tracked defect species as a function of position and time
$\boldsymbol{J}$	defect flux	Flow of defect concentration through the material
R	reaction term	Net local source or sink due to annealing, recombination, clustering, or related kinetics
D	diffusion coefficient	Parameter controlling the rate of defect spreading in space
k_{ann}	annealing rate constant	First-order rate constant for removal of the tracked defect population
x, y	spatial coordinates	In-plane coordinates for the 2D computational domain
t	time	Evolution time
∇	gradient operator	Spatial derivative operator $\left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}\right)$ in 2D

Symbol	Name	Meaning
∇^2	Laplacian operator	Sum of second spatial derivatives, $\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$, in 2D Cartesian coordinates
$\boldsymbol{n}$	outward unit normal	Unit vector normal to the boundary
$C_0(x, y)$	initial condition	Defect concentration field at $t = 0$
$M(t)$	total defect mass	Domain integral of defect concentration; conserved in pure diffusion with zero-flux boundaries
T	temperature	Local temperature field, later supplied by Module 4
D_0	diffusion prefactor	Pre-exponential factor in the Arrhenius diffusion model
k_0	annealing prefactor	Pre-exponential factor in the Arrhenius annealing model
E_D	diffusion activation energy	Energy barrier for defect migration
E_a	annealing activation energy	Energy barrier for annealing kinetics
k_B	Boltzmann constant	Constant connecting temperature to thermal energy
N_x, N_y	grid counts	Number of grid points in the x- and y-directions
$\Delta x, \Delta y$	grid spacings	Spatial step sizes of the structured 2D grid
$C^n_{i,j}$	discrete concentration	Numerical approximation to C at grid point (i, j) and time step n
Δt	time step	Numerical time increment between updates
∇^2_h	discrete Laplacian	Finite-difference approximation to the continuous Laplacian operator

14 Immediate implementation recommendation

The next coding step should be to implement the 2D Module 3 skeleton exactly at the level described in this note, run the verification cases, and only then connect the solver to imported damage maps from Module 1. Once that baseline is stable, the project can move to 2D Module 2 electrostatics with a clean defect-field interface already in place.